

Практикум студента

Согласование изменений и проверка версии

Цель: сохранить две независимые работы, объяснить отклонение push, опубликовать проверенную версию и восстановить её историю.

Сначала слушайте объяснение по слайдам, затем выполняйте шаги самостоятельно. На работу предусмотрено 60 минут. Все команды ниже вводятся в Git Bash, по одной строке; символ \$ вводить не нужно. Пример вывода не является командой.

Подготовка Windows 10 и 11

Скачайте 03_Стартовый_проект.zip. В Проводнике выберите «Извлечь всё». Откройте папку git-remote-p2. Работайте в отдельной новой папке внутри своих Документов; не распаковывайте поверх старой попытки. Запустите 00_OPEN_GIT_BASH.cmd.

Резервный запуск: правая кнопка по свободному месту папки → Open Git Bash here; в Windows 11 может понадобиться «Показать дополнительные параметры». Команды не вводятся в PowerShell, командной строке Windows или Блокноте. Файлы .sh запускайте через bash из Git Bash, как написано в шагах.

Если Git ещё не установлен: <https://git-scm.com/install/windows>. Выберите установщик для архитектуры компьютера, завершите установку и заново откройте Git Bash. Тип системы указан в Параметры → Система → О системе. Если установка недоступна, сообщите преподавателю: во время пары заполните объяснения и прогнозы, техническую часть завершите позднее. Это промежуточная, а не полная сдача.

Для вставки команды используйте Shift+Insert или меню правой кнопки. Кавычки должны быть прямыми. После случайного незакрытого выражения нажмите Ctrl+C и повторите одну строку. Перед каждым блоком сверяйте начальную папку; cd изменяет её для всех следующих команд.

Учебный сервис не запускается. Файлы config и README моделируют его конфигурацию. Общий репозиторий находится на том же диске: это модель обмена, не опыт сетевой отказоустойчивости.

Б1 Запуск независимого стенда

Начальная папка: *git-remote-p2*. Все строки выполняйте по порядку в одном окне *Git Bash*.

```
pwd
git --version
bash setup.sh
git -C alice status -sb
git -C bob status -sb
```

Ожидаемый результат

Созданы *team.git*, *alice* и *bob*. Обе копии на *main*, исходный тег *v0.2.0*. В каждой один подготовленный коммит.

Если результат отличается

setup.sh рассчитан на новую папку. Если рабочие каталоги уже существуют, он остановится и сохранит их. Для новой попытки распакуйте ZIP отдельно.

Б2 Согласованная правка конфигурации

Начальная папка: *git-remote-p2*. Все строки выполняйте по порядку в одном окне *Git Bash*.

```
cd alice
git config user.name "Student Alice"
git config user.email "alice@example.invalid"
printf "interval=30\n" > config.txt
printf "Service configuration\nMonitoring interval: 30 seconds.\n" > README.txt
bash check.sh
git diff -- config.txt README.txt
git add config.txt README.txt
git commit -m "Set and document interval 30"
git push origin main
```

Ожидаемый результат

Проверка выводит `PASS: interval=30; documentation matches`. Новая версия отправлена; bob ещё на исходной версии 60.

Если результат отличается

Если `FAIL`, сравните две строки из условия: значение `config` и число в `README` должны совпадать. Не делайте `commit` до исправления.

В командах `user.name` замените `Student` своим именем, сохраняя `Alice` или `Bob` как роль. Адрес `example.invalid` — учебный пример. Не указывайте пароль, токен или данные чужой учётной записи.

Б3 Независимая работа второй роли

Начальная папка: *alice*. Все строки выполняйте по порядку в одном окне *Git Bash*.

```
cd ../bob
git config user.name "Student Bob"
git config user.email "bob@example.invalid"
cat config.txt
printf "v0.3.0\nConfiguration and documentation updated.\n" > CHANGELOG.txt
git add CHANGELOG.txt
git commit -m "Add release notes"
git push origin main > ../evidence/rejected-push.txt 2>&1
cat ../evidence/rejected-push.txt
```

Ожидаемый результат

До отправки config в bob равен 60. Push отклонён: rejected, fetch first или non-fast-forward. Это запланированный результат.

Если результат отличается

Не выполняйте pull до этого шага и не повторяйте push с --force. Если push прошёл, проверьте, был ли ранее успешный push из *alice*. Отсутствие отказа честно опишите в отчёте.

В командах user.name замените Student своим именем, сохраняя Alice или Bob как роль. Адрес example.invalid — учебный пример. Не указывайте пароль, токен или данные чужой учётной записи.

Символы > и 2>&1 сохраняют сообщения команды в файл. Ненулевой код этой отправки ожидается. Следующий cat показывает сохранённое сообщение. До Б4 не выполняйте повторный push или pull.

Б4 Получение и согласование историй

Начальная папка: bob. Все строки выполняйте по порядку в одном окне Git Bash.

```
git fetch origin
git --no-pager log --oneline --graph --all --decorate
git --no-pager log --oneline main..origin/main
git --no-pager log --oneline origin/main..main
git merge --no-edit origin/main
bash check.sh
cat CHANGELOG.txt
git rev-list --parents -n 1 HEAD
git status -sb
```

Ожидаемый результат

Слияние проходит автоматически: менялись разные файлы. PASS для 30; CHANGELOG сохранён; rev-list показывает хеш merge и два хеша родителей.

Если результат отличается

Если появился CONFLICT, не ставьте тег. Проверьте diff и status с преподавателем. Автоматическое слияние ожидается только при точном выполнении шагов; не переписывайте файл вслепую.

Локальная main и origin/main содержат по одному собственному коммиту. В точном маршруте merge проходит без текстового конфликта. Отсутствие конфликта не заменяет запуск check.sh.

Б5 Публикация проверенной версии

Начальная папка: bob. Все строки выполняйте по порядку в одном окне Git Bash.

```
git tag -a v0.3.0 -m "Verified shared configuration"
git push origin main v0.3.0
git ls-remote --tags origin
```

Ожидаемый результат

В team.git отправлены main и тег v0.3.0. Для аннотированного тега ls-remote может показать две строки: объект тега и коммит.

Если результат отличается

Тег уже существует: сначала git show --stat v0.3.0. Не переносите тег принудительно. Если проверка не пройдена, вернитесь к Б4.

Б6 Получение версии первой ролью

Начальная папка: bob. Все строки выполняйте по порядку в одном окне Git Bash.

```
cd ../alice
git pull --ff-only origin main
git fetch origin --tags
bash check.sh
git rev-parse HEAD
git -C ../bob rev-parse HEAD
git --git-dir=../team.git rev-parse main
git rev-parse "v0.3.0^{commit}"
```

Ожидаемый результат

Все четыре хеша совпадают; обе копии на одной проверенной версии. В точном маршруте четыре коммита, включая merge.

Если результат отличается

pull --ff-only остановится при новых локальных коммитах; это защита от неожиданного слияния. Сначала выясните причину расхождения с преподавателем.

Б7 Упаковка и восстановление

Начальная папка: *alice*. Все строки выполняйте по порядку в одном окне *Git Bash*.

```
cd ..  
bash pack.sh  
git clone submission/work.bundle restored  
git -C restored status -sb  
git -C restored rev-parse "v0.3.0^{commit}"  
bash restored/check.sh
```

Ожидаемый результат

В *submission* есть *release.zip* и переносимая история. В *restored* проверка *PASS*; тег совпадает с выпущенной версией.

Если результат отличается

Папка *restored* должна быть новой. Если она уже существует, проверьте старую копию или выберите *restored2* и замените имя во всех командах. *check.sh* сам переходит к своей папке.

Проверка в *restored* обязательна для полной работы. Она подтверждает переносимость *bundle*. При повторном запуске *pack.sh* заполненный *submission/report.txt* сохраняется.

Отчёт и последующая сдача

1. В Проводнике откройте submission/report.txt через «Открыть с помощью» → Блокнот. Заполните все поля своими наблюдениями и нажмите Ctrl+S. Если этап не выполнен, укажите это прямо. Нельзя подставлять вымышленные хеши или сообщения.
2. Включите расширения файлов: Windows 10 → вкладка «Вид»; Windows 11 → «Вид» → «Показать» → «Расширения имён файлов». Проверьте, что отчёт называется report.txt, а не report.txt.txt.
3. Выделите содержимое submission: work.bundle, history.txt, state.txt, report.txt и папка evidence, а также release.zip. Windows 11: «Сжать в ZIP-файл»; Windows 10: «Отправить» → «Сжатая ZIP-папка». Назовите внешний архив Фамилия_Согласование_Git.zip.
4. Откройте созданный ZIP и проверьте вложения. Сохраните архив и рабочие папки до проверки. Позднее загрузите один ZIP в задание ЭОС для соответствующей пары, когда преподаватель объявит ссылку и срок. Во время пары загрузка не требуется.

Оценивание из 10 баллов

Два независимых изменения и свидетельство отклонённого push	2
Обе работы объединены и сохранены в истории	2
Конфигурация проверена; границы теста объяснены	2
Тег опубликован, обе копии получили общий коммит	2
Выпуск, bundle, восстановление и отчёт представлены	2

По каждому критерию: 2 — полностью и верно; 1 — частично; 0 — отсутствует или неверно. Дополнительные осмысленные исправления объясните; число коммитов не является самостоятельным критерием.

Помощь и дополнительное наблюдение

Неверная папка

Выполните `pwd`. До перехода в `alice` или `bob` нахождение в корневой папке стенда нормально. Внутри `team.git` рабочие файлы не редактируют.

Автор не настроен

В текущей рабочей копии задайте `user.name` и `user.email`. Затем повторите только неудавшийся `commit`.

Папка уже существует

Вероятно, часть работы выполнена. Посмотрите состояние. Для новой попытки используйте другую папку; `.git` не удаляйте.

Рабочее дерево изменено

Посмотрите `git status` и `git diff`. Перед слиянием или упаковкой выясните, что изменилось. Не выполняйте `reset --hard` или очистку наугад.

Команда зависла в просмотре

Если `log` открыл просмотрщик, нажмите `q`. В практикуме используется `--no-pager`, чтобы вывод оставался в терминале.

Проверка не пройдена

Сообщите номер шага, `pwd`, точную команду и весь ответ Git. Сохраните папку. Не продолжайте выпуск при незавершённом слиянии.

Для завершивших раньше

Только в `restored` измените `config.txt` на `interval=0` и запустите `bash check.sh`. Ожидается FAIL. Затем `git restore config.txt` внутри `restored` и повторная проверка PASS. Основные `alice` и `bob` не изменяйте. Объясните, почему успешный тест не доказывает работу настоящего сервиса.

Официальная справка: <https://git-scm.com/docs/git-push> и <https://git-scm.com/docs/git-merge>